

Teoría de la Información y Codificación

Práctica 3: Huffman como algoritmo de compresión

José A. Montenegro Montes

5 de febrero de 2026

1. Enunciado

En esta práctica utilizaremos el algoritmo de Huffman para comprimir, basándonos en la codificación por símbolos.

Esta práctica será de utilidad para comparar posteriormente los algoritmos de compresión en bloques.

La práctica está basada en el código del libro Algorithms 4 Edición (Autores Robert Sedgewick y Kevin Wayne), que puede ser consultado en biblioteca.

2. Conclusiones

Además de realizar la comparativa con los códigos en bloques, la práctica ofrece un ejemplo de lectura y escritura binaria y cómo es posible utilizar la librería diffutils.

El algoritmo es aplicado a los tres libros disponibles en el campus virtual: **Alice.txt**, **MobyDick.txt** y **Quixote.txt**. La práctica genera el archivo comprimido con el sufijo C.txt y el archivo descomprimido con el sufijo D.txt. Los sufijos pueden ser modificados en el código. Por ejemplo, con la ejecución sobre el libro Alice.txt obtendremos Alice.txtC.txt como archivo comprimido y Alice.txtD.txt como archivo descomprimido.

Además de utilizar los libros anteriormente citados, sería interesante aplicar la práctica a un **documento aleatorio** y comparar su salida posteriormente con la codificación en bloques.

La salida del programa será la siguiente por defecto la siguiente:

```
1
2
3 ***** Alice.txt *****
4 Size regular: 152146 Size compress: 87816 Ratio: 0.5771825
5 Fichero Descomprimido correctamente!!! Size regular: 152146 Size decompress: 152146
6 Tiempo Comprimir: 65872000 nanosegundos. Tiempo Descomprimir: 56971000 nanosegundos.
7 *****
```

donde nos ofrecen información del ratio de compresión, el tiempo utilizado en comprimir y descomprimir. Además verificamos que el archivo descomprimido es igual al original.

Si ejecutamos la aplicación con la opción de estadísticas habilitada, nos mostrará la codificación utilizada, frecuencias y probabilidades de los caracteres del texto y finalmente la entropía y la longitud media obtenida.

```

1
2
3 ***** Alice.txt *****
4 Size regular: 152146 Size compress: 87816 Ratio: 0.5771825
5 Fichero Descomprimido correctamente!!! Size regular: 152146 Size decompress: 152146
6 Tiempo Comprimir: 65872000 nanosegundos. Tiempo Descomprimir: 56971000 nanosegundos.
7 *****
8
9
10 Caracter (Frecuencia - Probabilidad) Codificacion
11 -----
12 10 (3601 - 0.023668056) 01000
13 13 (3601 - 0.023668056) 01001
14 26 (1 - 6.572634E-6) 01011100110010010
15 32 (28917 - 0.19006087) 00
16 33 (450 - 0.0029576854) 111011011
17 34 (113 - 7.4270763E-4) 11101101010
18 39 (1763 - 0.011587554) 1110111
19 40 (56 - 3.680675E-4) 111011001111
20 41 (55 - 3.6149487E-4) 111011001110
21 42 (60 - 3.9435804E-4) 01011100100
22 44 (2418 - 0.015892629) 100111
23 45 (669 - 0.0043970924) 11000010
24 46 (978 - 0.0064280364) 0101011
25 48 (1 - 6.572634E-6) 01011100110011000
26 51 (1 - 6.572634E-6) 01011100110011001
27 58 (233 - 0.0015314238) 010101000
28 59 (194 - 0.0012750911) 1110110000
29 63 (202 - 0.0013276722) 1110110010
30 65 (638 - 0.0041933404) 10011011
31 66 (91 - 5.9810973E-4) 11000011001
32 67 (145 - 9.5303194E-4) 1001101001
33 68 (192 - 0.0012619458) 1100001111
34 69 (188 - 0.0012356553) 1100001110
35 70 (74 - 4.8637492E-4) 10011010100
36 71 (83 - 5.4552866E-4) 10011010111
37 72 (284 - 0.0018666281) 010111011
38 73 (733 - 0.004817741) 11000110
39 74 (8 - 5.2581072E-5) 0101110011000
40 75 (82 - 5.38956E-4) 10011010110
41 76 (98 - 6.441182E-4) 11101100110
42 77 (200 - 0.0013145269) 1110110001
43 78 (120 - 7.887161E-4) 0101110000
44 79 (176 - 0.0011567837) 1100001101
45 80 (65 - 4.272212E-4) 01011100101
46 81 (84 - 5.521013E-4) 11000011000
47 82 (140 - 9.2016876E-4) 0101110101
48 83 (218 - 0.0014328342) 1110110100

```

49	84	(472 - 0.0031022832)	01010101
50	85	(66 - 4.3379384E-4)	01011100111
51	86	(42 - 2.7605065E-4)	010111001101
52	87	(237 - 0.0015577143)	010101001
53	88	(4 - 2.6290536E-5)	010111001100101
54	89	(114 - 7.4928027E-4)	11101101011
55	90	(1 - 6.572634E-6)	01011100110010011
56	91	(2 - 1.3145268E-5)	0101110011001000
57	93	(2 - 1.3145268E-5)	0101110011001101
58	95	(4 - 2.6290536E-5)	010111001100111
59	96	(1109 - 0.007289051)	1001100
60	97	(8153 - 0.053586688)	0111
61	98	(1384 - 0.009096526)	1100000
62	99	(2254 - 0.014814718)	100001
63	100	(4739 - 0.031147713)	10010
64	101	(13386 - 0.08798128)	1101
65	102	(1927 - 0.012665466)	010110
66	103	(2448 - 0.016089808)	101001
67	104	(7090 - 0.046599977)	11111
68	105	(6781 - 0.044569034)	11100
69	106	(138 - 9.070235E-4)	0101110100
70	107	(1076 - 0.0070721544)	0101111
71	108	(4618 - 0.030352425)	10001
72	109	(1907 - 0.012534013)	010100
73	110	(6896 - 0.045324884)	11110
74	111	(7970 - 0.052383896)	0110
75	112	(1459 - 0.009589474)	1100010
76	113	(125 - 8.2157925E-4)	0101110001
77	114	(5297 - 0.034815244)	10101
78	115	(6282 - 0.04128929)	11001
79	116	(10217 - 0.067152604)	1011
80	117	(3402 - 0.022360101)	111010
81	118	(804 - 0.005284398)	11000111
82	119	(2438 - 0.016024083)	101000
83	120	(144 - 9.464593E-4)	1001101000
84	121	(2149 - 0.014124591)	100000
85	122	(77 - 5.0609285E-4)	10011010101
86			
87	Entropia:	4.5675883	
88	Longitud Media:	4.6123915	
89			

3. Código

Clase FuenteFile.

```
package Practica3;

import java.io.BufferedReader;
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
import java.util.LinkedList;
import java.util.List;
```

```

import difflib.Delta;
import difflib.DiffUtils;
import difflib.Patch;

public class FuenteFile {

    String path="/Users/";
    String compressSufix="C.txt";
    String decompressSufix="D.txt";
    Huffman huffman;
    String file;
    long nanoCompIni,nanoCompFin;
    long nanoDeCompIni,nanoDeCompFin;

    public FuenteFile(String file) throws FileNotFoundException{
        file=file;
        huffman = new Huffman ();

        nanoCompIni=System.nanoTime();
        huffman.compress(path+file,path+file+compressSufix);
        nanoCompFin=System.nanoTime();

        nanoDeCompIni=System.nanoTime();
        huffman.expand(path+file+compressSufix,path+file+decompressSufix);
        nanoDeCompFin=System.nanoTime();

    }
    /*****
     * Metodo para Diff
     * @param filename
     * @return
     */
    private static List<String> fileToLines(String filename) {
        List<String> lines = new LinkedList<String>();
        String line = "";
        BufferedReader in;
        try {
            in = new BufferedReader(new FileReader(filename));
            while ((line = in.readLine()) != null) {
                lines.add(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
        return lines;
    }
    /*****
     *
     */
    public void stadistic(boolean printCodeTable){
        long regularL,compressL,decompressL;
        float ratio;

        System.out.println("***** \t "+file+" \t *****");

        File regular = new File(path+file);
        regularL=regular.length();

```

```

File compress = new File(path+file+compressSufix);
compressL=compress.length();

ratio= (float) compressL / (float) regularL;
System.out.println("Size regular: "+regularL+" Size compress: "+compressL+ " Ratio: "+ ratio);

File decompress = new File(path+file+decompressSufix);
decompressL=decompress.length();

List<String> regularDiff = fileToLines(path+file);
List<String> decompressDiff = fileToLines(path+file+decompressSufix);

// Compute diff. Get the Patch object. Patch is the container for computed deltas.
Patch patch = DiffUtils.diff(regularDiff, decompressDiff);
List<Delta> diff= patch.getDeltas();

if (diff.isEmpty()) {
    System.out.print("Fichero Descomprimido correctamente!!!");
    System.out.println("\t Size regular: "+regularL+" Size decompress: "+decompressL);
    System.out.println("Tiempo Comprimir: "+(nanoCompFin-nanoCompIni)+" nanosegundos.
        Tiempo Descomprimir: "+(nanoDeCompFin-nanoDeCompIni)+" nanosegundos.");
}
else
    System.out.println("Error al Descomprimir!!!");

System.out.println("*****");

if (printCodeTable) huffman.printCodeTableAndFrequency();
}

public static void main(String[] args) {

    try {
        FuenteFile f1= new FuenteFile("Alice.txt");
        f1.stadistic(true);

        FuenteFile f2= new FuenteFile("MobyDick.txt");
        f2.stadistic(true);

        FuenteFile f3= new FuenteFile("Quixote.txt");
        f3.stadistic(true);

    } catch (FileNotFoundException e) {
        e.printStackTrace();
    }

}

}

```

Clase Huffman.

Ipackage Practica3;

```

import java.io.FileNotFoundException;

/*****
 * Codigo basado en http://algs4.cs.princeton.edu/55compression/
 *
 *****/

public class Huffman {

    // alphabet size of extended ASCII
    private static final int R = 256;
    BinaryIn In;
    BinaryOut Out;
    String[] codeTable;
    int[] freq;

    public Huffman(){

    }

    // Huffman trie node
    private class Node implements Comparable<Node> {
        private final char ch;
        private final int freq;
        private final Node left, right;

        Node(char ch, int freq, Node left, Node right) {
            this.ch = ch;
            this.freq = freq;
            this.left = left;
            this.right = right;
        }

        // is the node a leaf node?
        private boolean isLeaf() {
            assert (left == null && right == null) || (left != null && right != null);
            return (left == null && right == null);
        }

        // compare, based on frequency
        public int compareTo(Node that) {
            return this.freq - that.freq;
        }
    }

    /*****
     * Compress
     * @param fileIn
     * @param fileOut
     * @throws FileNotFoundException
     */
    // compress bytes from standard input and write to standard output
    public void compress(String fileIn, String fileOut) throws FileNotFoundException{
        In = new BinaryIn(fileIn);
        Out = new BinaryOut(fileOut);

        // read the input

```

```

String s = In.readString();
char[] input = s.toCharArray();

// tabulate frequency counts
freq = new int[R];
for (int i = 0; i < input.length; i++)
    freq[input[i]]++;

// build Huffman trie
Node root = buildTrie(freq);

// build code table
codeTable = new String[R];
buildCode(codeTable, root, "");

// print trie for decoder
writeTrie(root);

// print number of bytes in original uncompressed message
Out.write(input.length);

// use Huffman code to encode input
for (int i = 0; i < input.length; i++) {
    String code = codeTable[input[i]];
    for (int j = 0; j < code.length(); j++) {
        if (code.charAt(j) == '0') {
            Out.write(false);
        }
        else if (code.charAt(j) == '1') {
            Out.write(true);
        }
        else throw new IllegalStateException("Illegal state");
    }
}

// close output stream
Out.close();

}

/*****
 * Imprime la codificacion
 */
public void printCodeTableAndFrequency(){
    long total=0;
    float prob[] = new float [R];

    System.out.println("\n\nCaracter  (Frecuencia - Probabilidad) \t Codificacion\n-----");

    for (int i=0;i<R;i++){
        if (codeTable[i]!=null){
            total+=freq[i];
        }
    }

    for (int i=0;i<R;i++){
        if (codeTable[i]!=null){
            prob[i]=(float) freq[i]/ (float) total;

```

```

        System.out.println(i+"\t (" +freq[i]+" - "+prob[i]+" ) "+codeTable[i]);

    }
    else    prob[i]=0;
}

System.out.println("\nEntropia:      "+entropia(prob));
System.out.println("Longitud Media: "+longitudMedia(prob)+"\n");
}

/*****
 *
 * @param probabilidades
 * @return
 */
public float longitudMedia(float probabilidades[]) {
    float l = 0;

    for (int i = 0; i < R; i++) {
        if (probabilidades[i]!=0) l += probabilidades[i] * codeTable[i].length();
    }
    return l;
}

/*****
 * Entropia
 * @param probabilidades
 * @return
 */
public float entropia(float probabilidades[]) {
    float h = 0;

    for (int i = 0; i < R; i++) {
        if (probabilidades[i]!=0) h += probabilidades[i] * logBase2(1.0/probabilidades[i]);
    }
    return h;
}

/*****
 * Calculo logartimo base 2 mediante cambio de base
 * @param num
 * @return
 */
private double logBase2(double num) {
    return logCambioBase(num,2);
}

private double logCambioBase(double num, int b) {
    return Math.log(num)/Math.log(b);
}

/*****
 * build the Huffman trie given frequencies
 * @param freq
 * @return
 */

private Node buildTrie(int[] freq) {

    // initialize priority queue with singleton trees

```



```

        MinPQ<Node> pq = new MinPQ<Node>();
        for (char i = 0; i < R; i++)
            if (freq[i] > 0)
                pq.insert(new Node(i, freq[i], null, null));

        // merge two smallest trees
        while (pq.size() > 1) {
            Node left = pq.delMin();
            Node right = pq.delMin();
            Node parent = new Node('\0', left.freq + right.freq, left, right);
            pq.insert(parent);
        }
        return pq.delMin();
    }

    /*****
     * write bitstring-encoded trie to standard output
     * @param x
     */
    private void writeTrie(Node x) {
        if (x.isLeaf()) {
            Out.write(true);
            Out.write(x.ch, 8);
            return;
        }
        Out.write(false);
        writeTrie(x.left);
        writeTrie(x.right);
    }

    /*****
     * make a lookup table from symbols and their encodings
     *
     * @param st
     * @param x
     * @param s
     */
    private void buildCode(String[] st, Node x, String s) {
        if (!x.isLeaf()) {
            buildCode(st, x.left, s + '0');
            buildCode(st, x.right, s + '1');
        }
        else {
            st[x.ch] = s;
        }
    }

    /*****
     * Expand
     * @param fileIn
     * @param fileOut
     * @throws FileNotFoundException
     */
    // expand Huffman-encoded input from standard input and write to standard output
    public void expand(String fileIn, String fileOut) throws FileNotFoundException{
        In = new BinaryIn(fileIn);
        Out = new BinaryOut(fileOut);
    }

```

```

        // read in Huffman trie from input stream
        Node root = readTrie();

        // number of bytes to write
        int length = In.readInt();

        // decode using the Huffman trie
        for (int i = 0; i < length; i++) {
            Node x = root;
            while (!x.isLeaf()) {
                boolean bit = In.readBoolean();
                if (bit) x = x.right;
                else    x = x.left;
            }
            Out.write(x.ch, 8);
        }
        Out.close();
    }

    /**
     *
     * @return
     */
    private Node readTrie() {
        boolean isLeaf = In.readBoolean();
        if (isLeaf) {
            return new Node(In.readChar(), -1, null, null);
        }
        else {
            return new Node('\0', -1, readTrie(), readTrie());
        }
    }
}

```

Clase MinPQ.

```

package Practica3;

/*****
 * Compilation:  javac MinPQ.java
 * Execution:    java MinPQ < input.txt
 *
 * Generic min priority queue implementation with a binary heap.
 * Can be used with a comparator instead of the natural order.
 *
 * % java MinPQ < tinyPQ.txt
 * E A E (6 left on pq)
 *
 * We use a one-based array to simplify parent and child calculations.
 */
import java.util.Comparator;
import java.util.Iterator;

```

```

import java.util.NoSuchElementException;

/**
 * Codigo basado en http://algs4.cs.princeton.edu/55compression/
 *
 * The MinPQ class represents a priority queue of generic keys.
 * It supports the usual insert and delete-the-minimum
 * operations, along with methods for peeking at the minimum key,
 * testing if the priority queue is empty, and iterating through
 * the keys.
 *
 * <p>
 * This implementation uses a binary heap.
 * The insert and delete-the-minimum operations take
 * logarithmic amortized time.
 * The min, size, and is-empty operations take constant time.
 * Construction takes time proportional to the specified capacity or the number of
 * items used to initialize the data structure.
 *
 * <p>
 * For additional documentation, see Section 2.4 of
 * Algorithms, 4th Edition by Robert Sedgewick and Kevin Wayne.
 *
 * @author Robert Sedgewick
 * @author Kevin Wayne
 */
public class MinPQ<Key> implements Iterable<Key> {
    private Key[] pq;           // store items at indices 1 to N
    private int N;              // number of items on priority queue
    private Comparator<Key> comparator; // optional comparator

    /**
     * Initializes an empty priority queue with the given initial capacity.
     * @param initCapacity the initial capacity of the priority queue
     */
    public MinPQ(int initCapacity) {
        pq = (Key[]) new Object[initCapacity + 1];
        N = 0;
    }

    /**
     * Initializes an empty priority queue.
     */
    public MinPQ() {
        this(1);
    }

    /**
     * Initializes an empty priority queue with the given initial capacity,
     * using the given comparator.
     * @param initCapacity the initial capacity of the priority queue
     * @param comparator the order to use when comparing keys
     */
    public MinPQ(int initCapacity, Comparator<Key> comparator) {
        this.comparator = comparator;
        pq = (Key[]) new Object[initCapacity + 1];
        N = 0;
    }

    /**
     * Initializes an empty priority queue using the given comparator.

```

```

    * @param comparator the order to use when comparing keys
    */
    public MinPQ(Comparator<Key> comparator) { this(1, comparator); }

    /**
     * Initializes a priority queue from the array of keys.
     * Takes time proportional to the number of keys, using sink-based heap construction.
     * @param keys the array of keys
    */
    public MinPQ(Key[] keys) {
        N = keys.length;
        pq = (Key[]) new Object[keys.length + 1];
        for (int i = 0; i < N; i++)
            pq[i+1] = keys[i];
        for (int k = N/2; k >= 1; k--)
            sink(k);
        assert isMinHeap();
    }

    /**
     * Is the priority queue empty?
     * @return true if the priority queue is empty; false otherwise
    */
    public boolean isEmpty() {
        return N == 0;
    }

    /**
     * Returns the number of keys on the priority queue.
     * @return the number of keys on the priority queue
    */
    public int size() {
        return N;
    }

    /**
     * Returns a smallest key on the priority queue.
     * @return a smallest key on the priority queue
     * @throws java.util.NoSuchElementException if priority queue is empty
    */
    public Key min() {
        if (isEmpty()) throw new NoSuchElementException("Priority queue underflow");
        return pq[1];
    }

    // helper function to double the size of the heap array
    private void resize(int capacity) {
        assert capacity > N;
        Key[] temp = (Key[]) new Object[capacity];
        for (int i = 1; i <= N; i++) temp[i] = pq[i];
        pq = temp;
    }

    /**
     * Adds a new key to the priority queue.
     * @param x the key to add to the priority queue
    */
    public void insert(Key x) {
        // double size of array if necessary

```

```

        if (N == pq.length - 1) resize(2 * pq.length);

        // add x, and percolate it up to maintain heap invariant
        pq[++N] = x;
        swim(N);
        assert isMinHeap();
    }

    /**
     * Removes and returns a smallest key on the priority queue.
     * @return a smallest key on the priority queue
     * @throws java.util.NoSuchElementException if the priority queue is empty
     */
    public Key delMin() {
        if (isEmpty()) throw new NoSuchElementException("Priority queue underflow");
        exch(1, N);
        Key min = pq[N--];
        sink(1);
        pq[N+1] = null; // avoid loitering and help with garbage collection
        if ((N > 0) && (N == (pq.length - 1) / 4)) resize(pq.length / 2);
        assert isMinHeap();
        return min;
    }

    /*****
     * Helper functions to restore the heap invariant.
     *****/

    private void swim(int k) {
        while (k > 1 && greater(k/2, k)) {
            exch(k, k/2);
            k = k/2;
        }
    }

    private void sink(int k) {
        while (2*k <= N) {
            int j = 2*k;
            if (j < N && greater(j, j+1)) j++;
            if (!greater(k, j)) break;
            exch(k, j);
            k = j;
        }
    }

    /*****
     * Helper functions for compares and swaps.
     *****/

    private boolean greater(int i, int j) {
        if (comparator == null) {
            return ((Comparable<Key>) pq[i]).compareTo(pq[j]) > 0;
        }
        else {
            return comparator.compare(pq[i], pq[j]) > 0;
        }
    }

    private void exch(int i, int j) {

```

```

        Key swap = pq[i];
        pq[i] = pq[j];
        pq[j] = swap;
    }

    // is pq[1..N] a min heap?
    private boolean isMinHeap() {
        return isMinHeap(1);
    }

    // is subtree of pq[1..N] rooted at k a min heap?
    private boolean isMinHeap(int k) {
        if (k > N) return true;
        int left = 2*k, right = 2*k + 1;
        if (left <= N && greater(k, left)) return false;
        if (right <= N && greater(k, right)) return false;
        return isMinHeap(left) && isMinHeap(right);
    }

    /*****
     * Iterators
     *****/

    /**
     * Returns an iterator that iterates over the keys on the priority queue
     * in ascending order.
     * The iterator doesn't implement <tt>remove()</tt> since it's optional.
     * @return an iterator that iterates over the keys in ascending order
     */
    public Iterator<Key> iterator() { return new HeapIterator(); }

    private class HeapIterator implements Iterator<Key> {
        // create a new pq
        private MinPQ<Key> copy;

        // add all items to copy of heap
        // takes linear time since already in heap order so no keys move
        public HeapIterator() {
            if (comparator == null) copy = new MinPQ<Key>(size());
            else copy = new MinPQ<Key>(size(), comparator);
            for (int i = 1; i <= N; i++)
                copy.insert(pq[i]);
        }

        public boolean hasNext() { return !copy.isEmpty(); }
        public void remove() { throw new UnsupportedOperationException(); }

        public Key next() {
            if (!hasNext()) throw new NoSuchElementException();
            return copy.delMin();
        }
    }
}

```

Clase BinaryIn.

```
package Practica3;
/*****
 * Compilation:  javac BinaryStdIn.java
 * Execution:    java BinaryStdIn < input > output
 *
 * Supports reading binary data from standard input.
 *
 * % java BinaryStdIn < input.jpg > output.jpg
 * % diff input.jpg output.jpg
 */

import java.io.BufferedReader;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;

/**
 * <i>Binary standard input</i>. This class provides methods for reading
 * in bits from standard input, either one bit at a time (as a <tt>boolean</tt>),
 * 8 bits at a time (as a <tt>byte</tt> or <tt>char</tt>),
 * 16 bits at a time (as a <tt>short</tt>), 32 bits at a time
 * (as an <tt>int</tt> or <tt>float</tt>), or 64 bits at a time (as a
 * <tt>double</tt> or <tt>long</tt>).
 *
 * <p>
 * All primitive types are assumed to be represented using their
 * standard Java representations, in big-endian (most significant
 * byte first) order.
 *
 * <p>
 * The client should not intermix calls to <tt>BinaryStdIn</tt> with calls
 * to <tt>StdIn</tt> or <tt>System.in</tt>;
 * otherwise unexpected behavior will result.
 *
 * @author Robert Sedgewick
 * @author Kevin Wayne
 */
public final class BinaryIn {
    private BufferedReader in;
    private final int EOF = -1;    // end of file

    private int buffer;            // one character buffer
    private int N;                 // number of bits left in buffer

    // static initializer
    //static { fillBuffer(); }

    // don't instantiate
    public BinaryIn(String file) throws FileNotFoundException {
        in = new BufferedReader(new FileInputStream(file));
        fillBuffer();
    }

    private void fillBuffer() {
        try { buffer = in.read(); N = 8; }
    }
}
```

```

        catch (IOException e) { System.out.println("EOF"); buffer = EOF; N = -1; }
    }

    /**
     * Close this input stream and release any associated system resources.
     */
    public void close() {
        try {
            in.close();
        }
        catch (IOException e) {
            e.printStackTrace();
            throw new RuntimeException("Could not close BinaryStdIn");
        }
    }

    /**
     * Returns true if standard input is empty.
     * @return true if and only if standard input is empty
     */
    public boolean isEmpty() {
        return buffer == EOF;
    }

    /**
     * Read the next bit of data from standard input and return as a boolean.
     * @return the next bit of data from standard input as a <tt>boolean</tt>
     * @throws RuntimeException if standard input is empty
     */
    public boolean readBoolean() {
        if (isEmpty()) throw new RuntimeException("Reading from empty input stream");
        N--;
        boolean bit = ((buffer >> N) & 1) == 1;
        if (N == 0) fillBuffer();
        return bit;
    }

    /**
     * Read the next 8 bits from standard input and return as an 8-bit char.
     * Note that <tt>char</tt> is a 16-bit type;
     * to read the next 16 bits as a char, use <tt>readChar(16)</tt>
     * @return the next 8 bits of data from standard input as a <tt>char</tt>
     * @throws RuntimeException if there are fewer than 8 bits available on standard input
     */
    public char readChar() {
        if (isEmpty()) throw new RuntimeException("Reading from empty input stream");

        // special case when aligned byte
        if (N == 8) {
            int x = buffer;
            fillBuffer();
            return (char) (x & 0xff);
        }

        // combine last N bits of current buffer with first 8-N bits of new buffer
        int x = buffer;
        x <<= (8-N);
        int oldN = N;
        fillBuffer();
    }

```



```

        if (isEmpty()) throw new RuntimeException("Reading from empty input stream");
        N = oldN;
        x |= (buffer >>> N);
        return (char) (x & 0xff);
        // the above code doesn't quite work for the last character if N = 8
        // because buffer will be -1
    }

    /**
     * Read the next r bits from standard input and return as an r-bit character.
     * @param r number of bits to read.
     * @return the next r bits of data from standard input as a <tt>char</tt>
     * @throws RuntimeException if there are fewer than r bits available on standard input
     * @throws RuntimeException unless 1 <= r <= 16
     */
    public char readChar(int r) {
        if (r < 1 || r > 16) throw new RuntimeException("Illegal value of r = " + r);

        // optimize r = 8 case
        if (r == 8) return readChar();

        char x = 0;
        for (int i = 0; i < r; i++) {
            x <<= 1;
            boolean bit = readBoolean();
            if (bit) x |= 1;
        }
        return x;
    }

    /**
     * Read the remaining bytes of data from standard input and return as a string.
     * @return the remaining bytes of data from standard input as a <tt>String</tt>
     * @throws RuntimeException if standard input is empty or if the number of bits
     * available on standard input is not a multiple of 8 (byte-aligned)
     */
    public String readString() {
        if (isEmpty()) throw new RuntimeException("Reading from empty input stream");

        StringBuilder sb = new StringBuilder();
        while (!isEmpty()) {
            char c = readChar();
            sb.append(c);
        }
        return sb.toString();
    }

    /**
     * Read the next 16 bits from standard input and return as a 16-bit short.
     * @return the next 16 bits of data from standard input as a <tt>short</tt>
     * @throws RuntimeException if there are fewer than 16 bits available on standard input
     */
    public short readShort() {
        short x = 0;
        for (int i = 0; i < 2; i++) {
            char c = readChar();
            x <<= 8;
            x |= c;
        }
    }

```

```

    }
    return x;
}

/**
 * Read the next 32 bits from standard input and return as a 32-bit int.
 * @return the next 32 bits of data from standard input as a <tt>int</tt>
 * @throws RuntimeException if there are fewer than 32 bits available on standard input
 */
public int readInt() {
    int x = 0;
    for (int i = 0; i < 4; i++) {
        char c = readChar();
        x <<= 8;
        x |= c;
    }
    return x;
}

/**
 * Read the next r bits from standard input and return as an r-bit int.
 * @param r number of bits to read.
 * @return the next r bits of data from standard input as a <tt>int</tt>
 * @throws RuntimeException if there are fewer than r bits available on standard input
 * @throws RuntimeException unless 1 &le; r &le; 32
 */
public int readInt(int r) {
    if (r < 1 || r > 32) throw new RuntimeException("Illegal value of r = " + r);

    // optimize r = 32 case
    if (r == 32) return readInt();

    int x = 0;
    for (int i = 0; i < r; i++) {
        x <<= 1;
        boolean bit = readBoolean();
        if (bit) x |= 1;
    }
    return x;
}

/**
 * Read the next 64 bits from standard input and return as a 64-bit long.
 * @return the next 64 bits of data from standard input as a <tt>long</tt>
 * @throws RuntimeException if there are fewer than 64 bits available on standard input
 */
public long readLong() {
    long x = 0;
    for (int i = 0; i < 8; i++) {
        char c = readChar();
        x <<= 8;
        x |= c;
    }
    return x;
}

/**
 * Read the next 64 bits from standard input and return as a 64-bit double.

```

```

        * @return the next 64 bits of data from standard input as a <tt>double</tt>
        * @throws RuntimeException if there are fewer than 64 bits available on standard input
        */
        public double readDouble() {
            return Double.longBitsToDouble(readLong());
        }

        /**
         * Read the next 32 bits from standard input and return as a 32-bit float.
         * @return the next 32 bits of data from standard input as a <tt>float</tt>
         * @throws RuntimeException if there are fewer than 32 bits available on standard input
         */
        public float readFloat() {
            return Float.intBitsToFloat(readInt());
        }

        /**
         * Read the next 8 bits from standard input and return as an 8-bit byte.
         * @return the next 8 bits of data from standard input as a <tt>byte</tt>
         * @throws RuntimeException if there are fewer than 8 bits available on standard input
         */
        public byte readByte() {
            char c = readChar();
            byte x = (byte) (c & 0xff);
            return x;
        }
    }
}

```

Clase BinaryOut.

```

package Practica3;
/*****
 * Compilation:  javac BinaryStdOut.java
 * Execution:    java BinaryStdOut
 *
 * Write binary data to standard output, either one 1-bit boolean,
 * one 8-bit char, one 32-bit int, one 64-bit double, one 32-bit float,
 * or one 64-bit long at a time.
 *
 * The bytes written are not aligned.
 *
 *****/

import java.io.BufferedOutputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;

/**
 * <i>Binary standard output</i>. This class provides methods for converting
 * primitive type variables (<tt>boolean</tt>, <tt>byte</tt>, <tt>char</tt>,
 * <tt>int</tt>, <tt>long</tt>, <tt>float</tt>, and <tt>double</tt>)
 * to sequences of bits and writing them to standard output.
 * Uses big-endian (most-significant byte first).
 */

```

```

* <p>
* The client must <tt>flush()</tt> the output stream when finished writing bits.
* <p>
* The client should not intermixing calls to <tt>BinaryStdOut</tt> with calls
* to <tt>StdOut</tt> or <tt>System.out</tt>; otherwise unexpected behavior
* will result.
*
* @author Robert Sedgewick
* @author Kevin Wayne
*/
public final class BinaryOut {
    String file = null;
    private   BufferedOutputStream out;

    private   int buffer;      // 8-bit buffer of bits to write out
    private   int N;           // number of bits remaining in buffer

    // don't instantiate
    public BinaryOut(String file) throws FileNotFoundException {
        file = file;

        out = new BufferedOutputStream(new FileOutputStream(file));
    }

    /**
     * Write the specified bit to standard output.
     */
    private void writeBit(boolean bit) {
        // add bit to buffer
        buffer <<= 1;
        if (bit) buffer |= 1;

        // if buffer is full (8 bits), write out as a single byte
        N++;
        if (N == 8) clearBuffer();
    }

    /**
     * Write the 8-bit byte to standard output.
     */
    private void writeByte(int x) {
        assert x >= 0 && x < 256;

        // optimized if byte-aligned
        if (N == 0) {
            try { out.write(x); }
            catch (IOException e) { e.printStackTrace(); }
            return;
        }

        // otherwise write one bit at a time
        for (int i = 0; i < 8; i++) {
            boolean bit = ((x >>> (8 - i - 1)) & 1) == 1;
            writeBit(bit);
        }
    }

    // write out any remaining bits in buffer to standard output, padding with 0s

```

```

private void clearBuffer() {
    if (N == 0) return;
    if (N > 0) buffer <= (8 - N);
    try { out.write(buffer); }
    catch (IOException e) { e.printStackTrace(); }
    N = 0;
    buffer = 0;
}

/**
 * Flush standard output, padding 0s if number of bits written so far
 * is not a multiple of 8.
 */
public void flush() {
    clearBuffer();
    try { out.flush(); }
    catch (IOException e) { e.printStackTrace(); }
}

/**
 * Flush and close standard output. Once standard output is closed, you can no
 * longer write bits to it.
 */
public void close() {
    flush();
    try { out.close(); }
    catch (IOException e) { e.printStackTrace(); }
}

/**
 * Write the specified bit to standard output.
 * @param x the <tt>boolean</tt> to write.
 */
public void write(boolean x) {
    writeBit(x);
}

/**
 * Write the 8-bit byte to standard output.
 * @param x the <tt>byte</tt> to write.
 */
public void write(byte x) {
    writeByte(x & 0xff);
}

/**
 * Write the 32-bit int to standard output.
 * @param x the <tt>int</tt> to write.
 */
public void write(int x) {
    writeByte((x >>> 24) & 0xff);
    writeByte((x >>> 16) & 0xff);
    writeByte((x >>> 8) & 0xff);
    writeByte((x >>> 0) & 0xff);
}

/**
 * Write the r-bit int to standard output.

```

```

    * @param x the <tt>int</tt> to write.
    * @param r the number of relevant bits in the char.
    * @throws RuntimeException if <tt>r</tt> is not between 1 and 32.
    * @throws RuntimeException if <tt>x</tt> is not between 0 and 2<sup>r</sup> - 1.
    */
    public void write(int x, int r) {
        if (r == 32) { write(x); return; }
        if (r < 1 || r > 32) throw new RuntimeException("Illegal value for r = " + r);
        if (x < 0 || x >= (1 << r)) throw new RuntimeException("Illegal " + r + "-bit char = " + x);
        for (int i = 0; i < r; i++) {
            boolean bit = ((x >>> (r - i - 1)) & 1) == 1;
            writeBit(bit);
        }
    }

    /**
     * Write the 64-bit double to standard output.
     * @param x the <tt>double</tt> to write.
     */
    public void write(double x) {
        write(Double.doubleToRawLongBits(x));
    }

    /**
     * Write the 64-bit long to standard output.
     * @param x the <tt>long</tt> to write.
     */
    public void write(long x) {
        writeByte((int) ((x >>> 56) & 0xff));
        writeByte((int) ((x >>> 48) & 0xff));
        writeByte((int) ((x >>> 40) & 0xff));
        writeByte((int) ((x >>> 32) & 0xff));
        writeByte((int) ((x >>> 24) & 0xff));
        writeByte((int) ((x >>> 16) & 0xff));
        writeByte((int) ((x >>> 8) & 0xff));
        writeByte((int) ((x >>> 0) & 0xff));
    }

    /**
     * Write the 32-bit float to standard output.
     * @param x the <tt>float</tt> to write.
     */
    public void write(float x) {
        write(Float.floatToRawIntBits(x));
    }

    /**
     * Write the 16-bit int to standard output.
     * @param x the <tt>short</tt> to write.
     */
    public void write(short x) {
        writeByte((x >>> 8) & 0xff);
        writeByte((x >>> 0) & 0xff);
    }

```

```

/**
 * Write the 8-bit char to standard output.
 * @param x the <tt>char</tt> to write.
 * @throws RuntimeException if <tt>x</tt> is not between 0 and 255.
 */
public void write(char x) {
    if (x < 0 || x >= 256) throw new RuntimeException("Illegal 8-bit char = " + x);
    writeByte(x);
}

/**
 * Write the r-bit char to standard output.
 * @param x the <tt>char</tt> to write.
 * @param r the number of relevant bits in the char.
 * @throws RuntimeException if <tt>r</tt> is not between 1 and 16.
 * @throws RuntimeException if <tt>x</tt> is not between 0 and 2r - 1.
 */
public void write(char x, int r) {
    if (r == 8) { write(x); return; }
    if (r < 1 || r > 16) throw new RuntimeException("Illegal value for r = " + r);
    if (x < 0 || x >= (1 << r)) throw new RuntimeException("Illegal " + r + "-bit char = " + x);
    for (int i = 0; i < r; i++) {
        boolean bit = ((x >>> (r - i - 1)) & 1) == 1;
        writeBit(bit);
    }
}

/**
 * Write the string of 8-bit characters to standard output.
 * @param s the <tt>String</tt> to write.
 * @throws RuntimeException if any character in the string is not
 * between 0 and 255.
 */
public void write(String s) {
    for (int i = 0; i < s.length(); i++)
        write(s.charAt(i));
}

/**
 * Write the String of r-bit characters to standard output.
 * @param s the <tt>String</tt> to write.
 * @param r the number of relevant bits in each character.
 * @throws RuntimeException if r is not between 1 and 16.
 * @throws RuntimeException if any character in the string is not
 * between 0 and 2r - 1.
 */
public void write(String s, int r) {
    for (int i = 0; i < s.length(); i++)
        write(s.charAt(i), r);
}
}

```
